

Companion to Bird & de Moor, Algebra of Programming

§9. Dynamic Programming

§9.1. Theory

§9.1.1. Example: segmenting a list

- $(\mathsf{T}) = \text{concat}$
 $\mathsf{T} = [\text{nil}, \text{cat}]$ takes a non-empty first list, so T° cuts a non-empty prefix off a list every way, and folding with T flattens a list of segments
- $(\mathsf{h}) = \text{list}(\text{sum})$
 $\mathsf{h} = [\text{nil}, \text{cons}(\text{sum} \times 1)]$ puts a segment's sum in front, so folding with h sums every segment
- $\mathsf{H} = \text{partition list}(\text{sum})$
 H segments a list every way, then sums each segment; with R comparing the largest entry, M cuts the list so that the largest segment sum is as small as possible

(9.1.1b)

	x	h(x)
1	$\text{inl}(())$	$[\]$
2	$\text{inr}([c], [])$	$[c]$
3	$\text{inr}([a, b], [c])$	$[a + b, c]$
4	$\text{inr}([a], [b + c])$	$[a, b + c]$

- $\mathsf{H}([a, b, c], \mathsf{ys}) \iff \mathsf{ys} = [a + b + c] \vee \mathsf{ys} = [a, b + c] \vee \mathsf{ys} = [a + b, c] \vee \mathsf{ys} = [a, b, c]$
 H sends $[a, b, c]$ to the sums of its four segmentations and nothing else
- $\mathsf{T}^\circ \mathsf{F}(\mathsf{H}) \mathsf{h} = \mathsf{H}$
cut once, solve the rest, put the pieces back together

(9.1.1d)

$\mathsf{h} : \mathsf{FB} \rightarrow \mathsf{B}$ a map, $\mathsf{T} : \mathsf{FA} \rightarrow \mathsf{A}$ an F-algebra, $\mathsf{R} : \mathsf{B} \rightarrow \mathsf{B}$.

$\mathsf{H} \triangleq (\mathsf{T})^\circ (\mathsf{h}) : \mathsf{A} \rightarrow \mathsf{B}$, $\mathsf{M} \triangleq \frac{\mathsf{H}}{\mathsf{S}} \text{est}(\mathsf{R})$ the problem to be solved, $(\mu \mathsf{X} : \mathsf{G}(\mathsf{X}))$ as in $\triangleq \mu$.

$\mathsf{DPSetting}(\mathsf{h}, \mathsf{R}) \triangleq \text{Map}(\mathsf{h}) \wedge \text{MonoAlg}(\mathsf{h}, \mathsf{R}^\circ) \wedge \mathsf{R}^\circ \mathsf{R}^\circ \subseteq \mathsf{R}^\circ$

h is a map, h is monotonic on R° , and R° is transitive — assumed by every theorem below

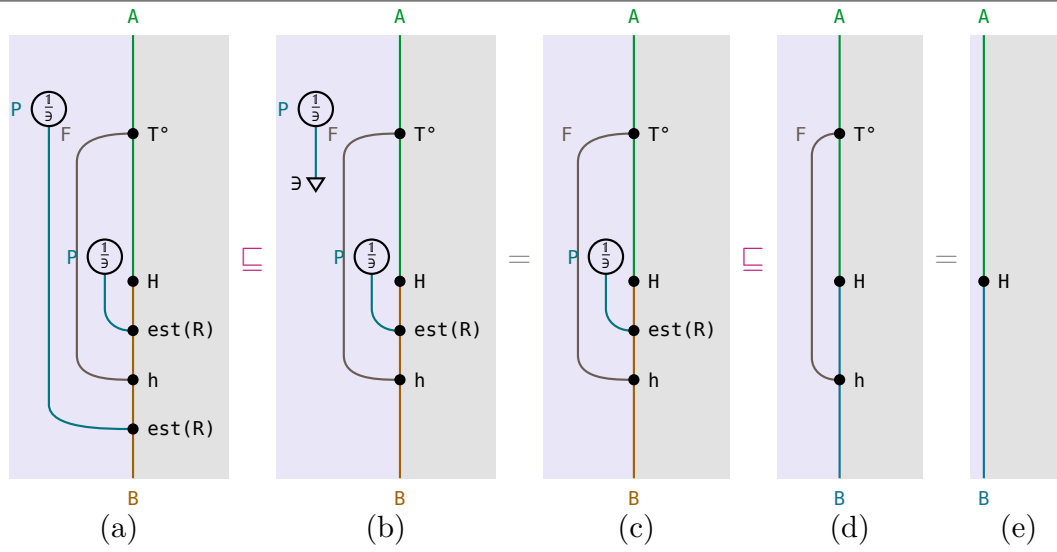
$(\mu \mathsf{X} : \frac{\mathsf{T}^\circ}{\mathsf{S}} \mathsf{P}(\mathsf{F}(\mathsf{X}) \mathsf{h}) \text{est}(\mathsf{R})) \subseteq \frac{\mathsf{H}}{\mathsf{S}} \text{est}(\mathsf{R})$
every answer the recursion returns is an optimal one

Theorem 9.1

- $\frac{\mathsf{T}^\circ}{\mathsf{S}}$ takes the input apart one step every way; $\mathsf{F}(\mathsf{X})$ solves each part by the recursion X ; h assembles each candidate; $\text{est}(\mathsf{R})$ keeps a best one
- $\sqsubseteq \mathsf{M}$: that never misses the optimum, though all candidates are never listed as H would
- h monotonic on R : a better part never makes the whole worse
- $\sqsubseteq$, not $=$: some optima may not be returned; that the recursion returns anything at all needs T° to stop taking apart — dynamic programming theorem

$$T^\circ F(H) h = H \implies \frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R)) h) \text{est}(R) \sqsubseteq H \quad (9.2)$$

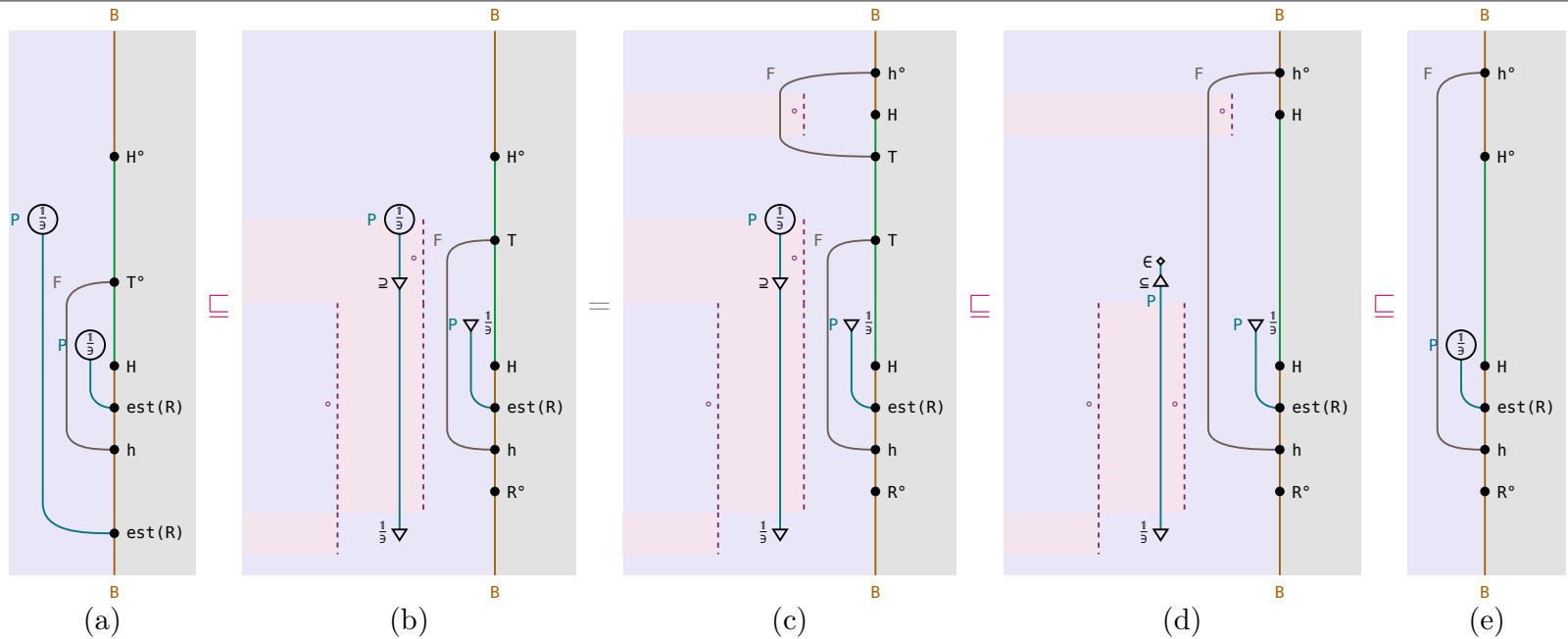
taking the input apart every way T allows ($\frac{T^\circ}{\exists}$), solving each part by M and keeping an optimum ($P(F(M)h) \text{est}(R)$)
returns only what H returns



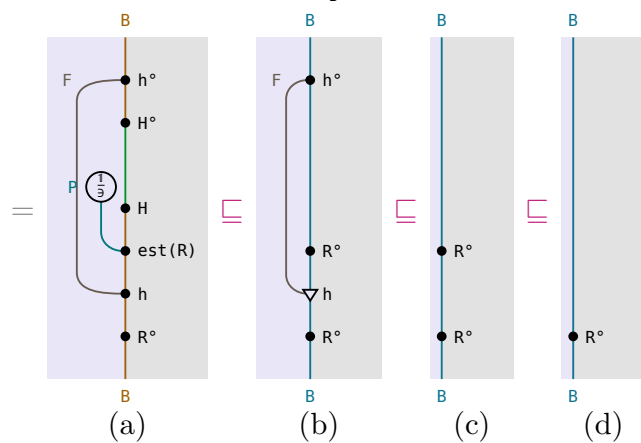
(a)	$\frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R)) h) \text{est}(R)$	$P(S) \text{est}(R) \sqsubseteq \exists S$
(b)	$\sqsubseteq \frac{T^\circ}{\exists} \exists F(\frac{H}{\exists} \text{est}(R)) h$	$\frac{R}{\exists} \exists = R$
(c)	$= T^\circ F(\frac{H}{\exists} \text{est}(R)) h$	$\frac{R}{\exists} \text{est}(R) \sqsubseteq S$
(d)	$\sqsubseteq T^\circ F(H) h$	$T^\circ F(H) h = H$
(e)	$= H$	

$$T^\circ F(H) h = H \implies H^\circ \frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R)) h) \text{est}(R) \sqsubseteq R^\circ \quad (9.3)$$

for every $\mathbf{b}$ that $\mathbf{H}$ returns from an input, the step $\frac{T^\circ}{\exists} P(F(\mathbf{M})h) \text{est}(R)$ returns from that input only $\mathbf{b}'$ with $\mathbf{R}$ relating $\mathbf{b}'$ to $\mathbf{b}$



(a)	$H^\circ \frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R)) h) \text{est}(R)$	$P(S) \text{est}(R) \sqsubseteq \epsilon \setminus (S R^\circ)$
(b)	$\sqsubseteq H^\circ \frac{T^\circ}{\exists} (\epsilon \setminus (F(\frac{H}{\exists} \text{est}(R)) h R^\circ))$	$T^\circ F(H) h = H$ $\implies H^\circ = h^\circ F(H^\circ) T$
(c)	$= h^\circ F(H^\circ) T \frac{T^\circ}{\exists} (\epsilon \setminus (F(\frac{H}{\exists} \text{est}(R)) h R^\circ))$	$T \frac{T^\circ}{\exists} \sqsubseteq \epsilon$
(d)	$\sqsubseteq h^\circ F(H^\circ) \epsilon (\epsilon \setminus (F(\frac{H}{\exists} \text{est}(R)) h R^\circ))$	$S (S \setminus R) \sqsubseteq R$
(e)	$\sqsubseteq h^\circ F(H^\circ) F(\frac{H}{\exists} \text{est}(R)) h R^\circ$	$t(f g) = t(f) t(g)$



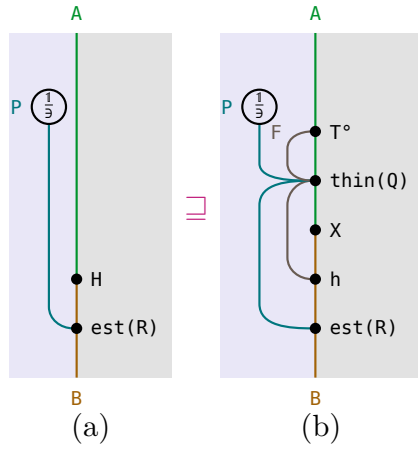
(a)	$= h^\circ F(H^\circ \frac{H}{\exists} \text{est}(R)) h R^\circ$	$S^\circ \frac{S}{\exists} \text{est}(R) \sqsubseteq R^\circ$
(b)	$\sqsubseteq h^\circ F(R^\circ) h R^\circ$	$\text{MonoAlg}(h, R^\circ)$
(c)	$\sqsubseteq R^\circ R^\circ$	$R^\circ R^\circ \sqsubseteq R^\circ$
(d)	$\sqsubseteq R^\circ$	

$$\text{ThinCondition}(T, h, R, Q) \Rightarrow (\mu X : \frac{T^\circ}{3} \text{thin}(Q) P(F(X) h) \text{est}(R)) \sqsubseteq \frac{H}{3} \text{est}(R)$$

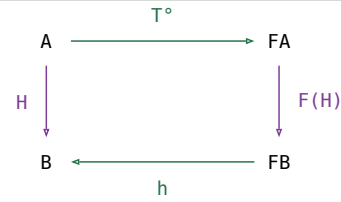
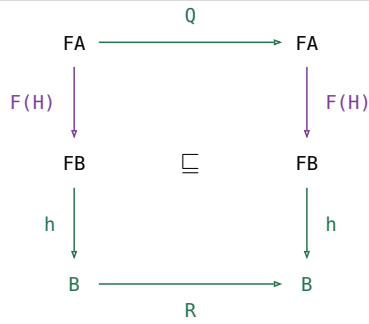
Theorem 9.2

an optimum over everything H returns is reached by taking the input apart every way T allows, dropping the parts that can never win, solving each of the rest and keeping one optimum

$$H \triangleq (T)^\circ (h) \triangleq H$$



(a)	$\frac{H}{3} \text{est}(R)$	$QF(H) h \sqsubseteq F(H) h R, h \text{ monotonic on } R \text{ — (8.1a.6), (7.1.1a)}$
(b)	$\sqsubseteq \frac{T^\circ}{3} \text{thin}(Q) P(F(X) h) \text{est}(R)$	



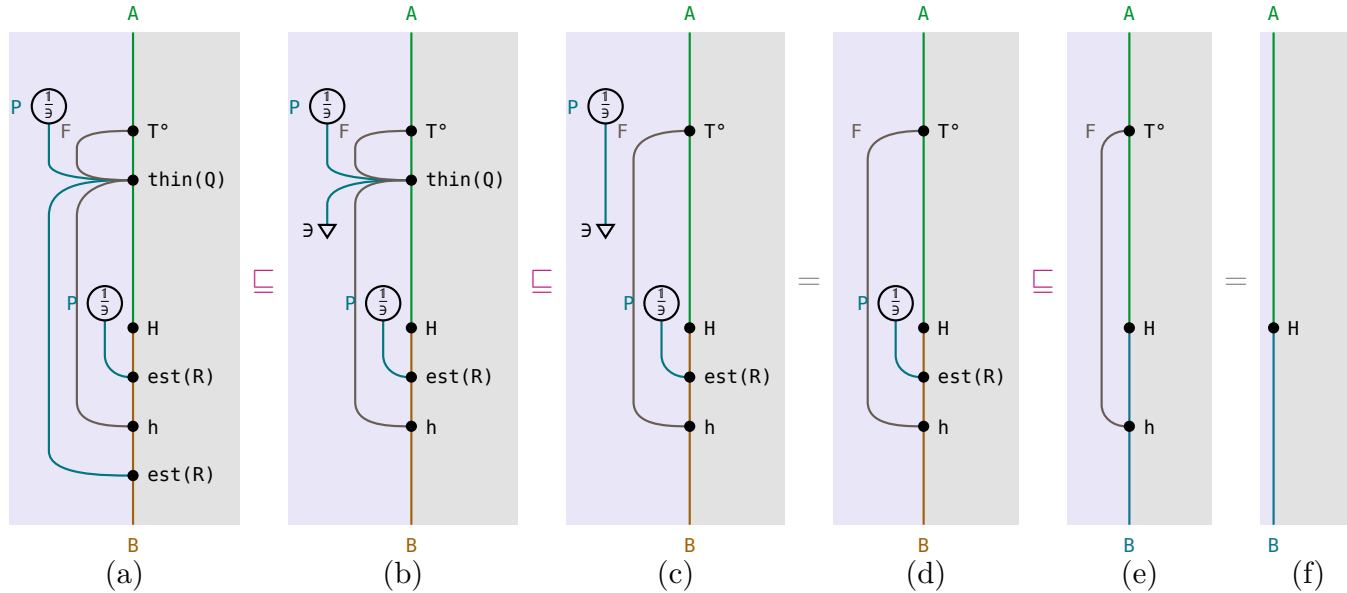
$T^\circ F(H) h = H$: taking the input apart once by T° , solving the parts by $F(H)$ and assembling by h returns exactly what H returns

$QF(H) h \sqsubseteq F(H) h R$: every result of thinning by Q , solving the parts by $F(H)$ and assembling by h is reached by R from a result of solving by $F(H)$ and assembling by h without Q

$$T^\circ F(H) h = H \Rightarrow \frac{T^\circ}{3} \text{thin}(Q) P(F(\frac{H}{3} \text{est}(R)) h) \text{est}(R) \sqsubseteq H$$

(9.1.1e)

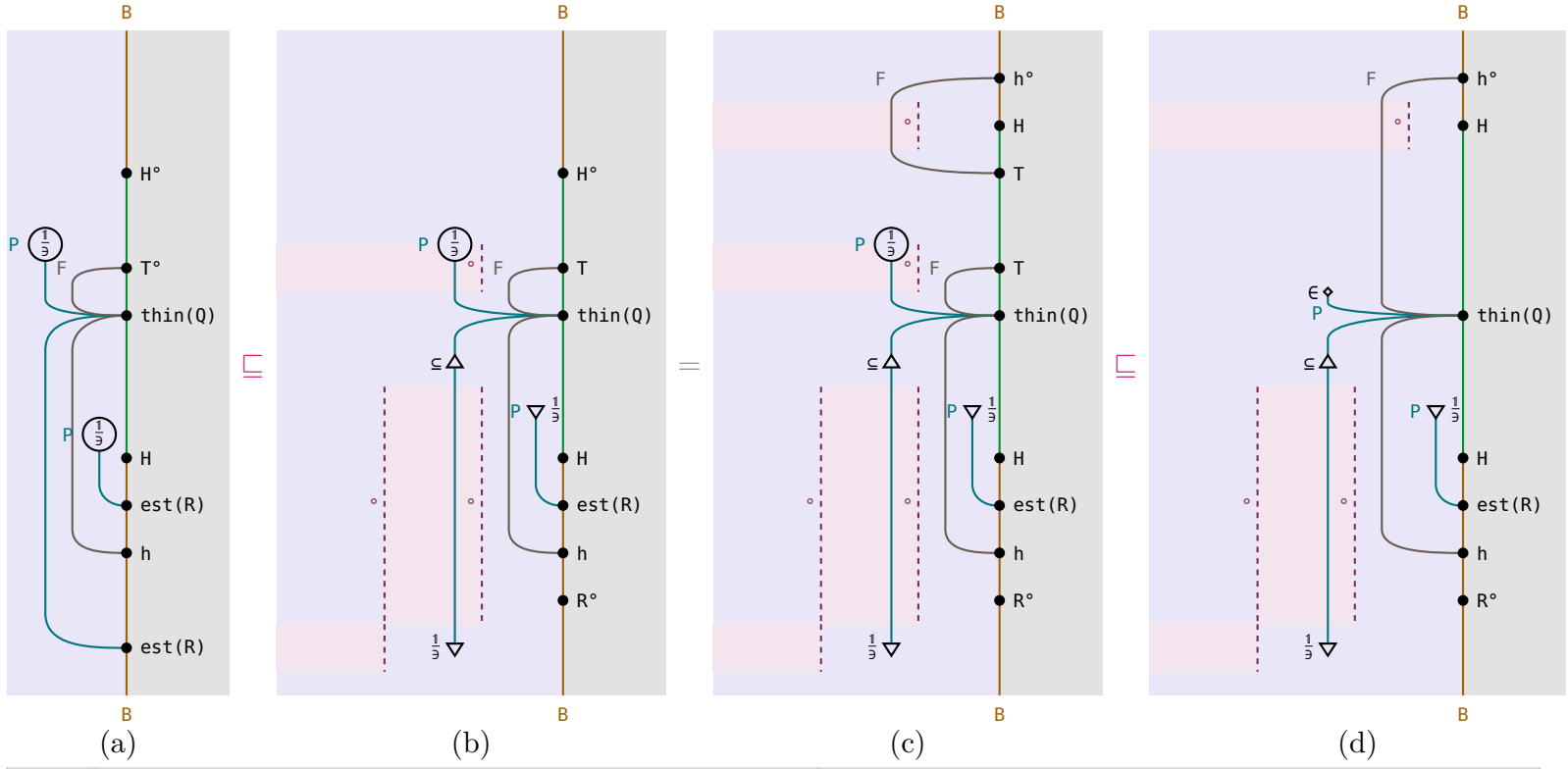
taking the input apart every way T allows ($\frac{T^\circ}{3}$), dropping the parts $\text{thin}(Q)$ rejects, solving each rest by M and keeping an optimum returns only what H returns



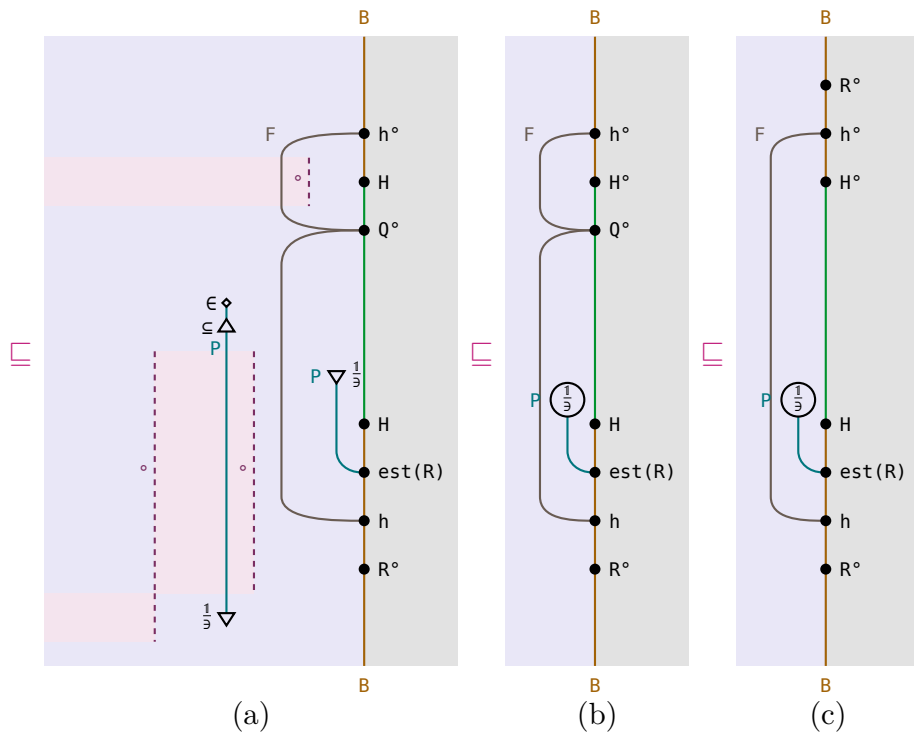
(a)	$\frac{T^\circ}{3} \text{thin}(Q) P(F(\frac{H}{3} \text{est}(R)) h) \text{est}(R)$	$P(S) \text{est}(R) \sqsubseteq \exists S$
(b)	$\sqsubseteq \frac{T^\circ}{3} \text{thin}(Q) \exists F(\frac{H}{3} \text{est}(R)) h$	$\text{thin}(Q) \exists \sqsubseteq \exists$
(c)	$\sqsubseteq \frac{T^\circ}{3} \exists F(\frac{H}{3} \text{est}(R)) h$	$\frac{R}{3} \exists = R$
(d)	$= T^\circ F(\frac{H}{3} \text{est}(R)) h$	$\frac{R}{3} \text{est}(R) \sqsubseteq S$
(e)	$\sqsubseteq T^\circ F(H) h$	$T^\circ F(H) h = H$
(f)	$= H$	

$$T^\circ F(H) h = H \wedge Q F(H) h \subseteq F(H) h R \implies H^\circ \frac{T^\circ}{\exists} \text{thin}(Q) P(F(\frac{H}{\exists} \text{est}(R)) h) \text{est}(R) \subseteq R^\circ \quad (9.1.1f)$$

H° followed by the body at M is ER° : an answer of the body is never worse than an answer of H to the same input

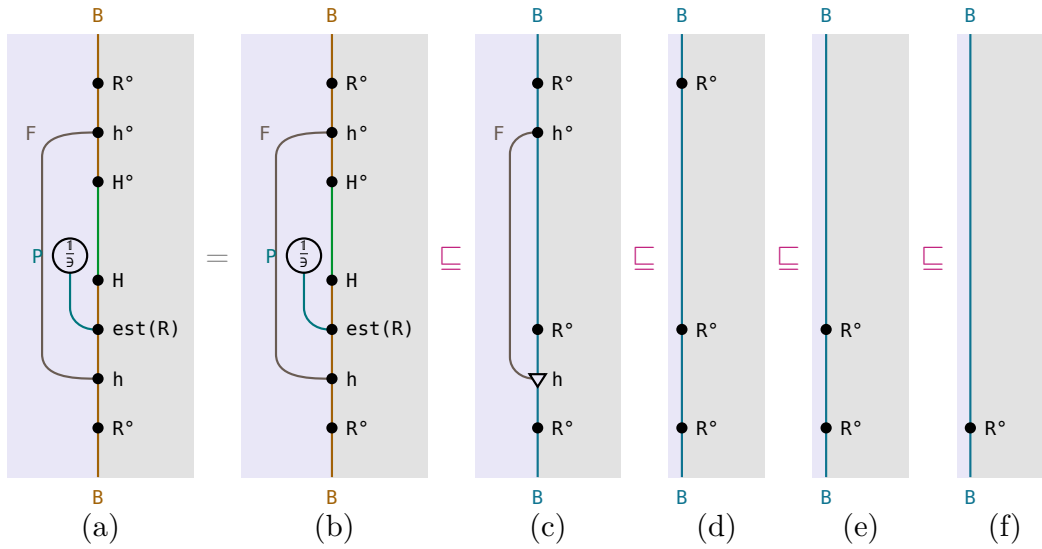


(a)	$H^\circ \frac{T^\circ}{\exists} \text{thin}(Q) P(F(\frac{H}{\exists} \text{est}(R)) h) \text{est}(R)$	$P(S) \text{est}(R) \subseteq E \setminus (S R^\circ)$
(b)	$\sqsubseteq H^\circ \frac{T^\circ}{\exists} \text{thin}(Q) (E \setminus (F(\frac{H}{\exists} \text{est}(R)) h R^\circ))$	$T^\circ F(H) h = H$ $\implies H^\circ = h^\circ F(H^\circ) T$
(c)	$= h^\circ F(H^\circ) T \frac{T^\circ}{\exists} \text{thin}(Q) (E \setminus (F(\frac{H}{\exists} \text{est}(R)) h R^\circ))$	$T \frac{T^\circ}{\exists} \subseteq E$
(d)	$\sqsubseteq h^\circ F(H^\circ) E \text{thin}(Q) (E \setminus (F(\frac{H}{\exists} \text{est}(R)) h R^\circ))$	$E \text{thin}(Q) \subseteq Q^\circ E$



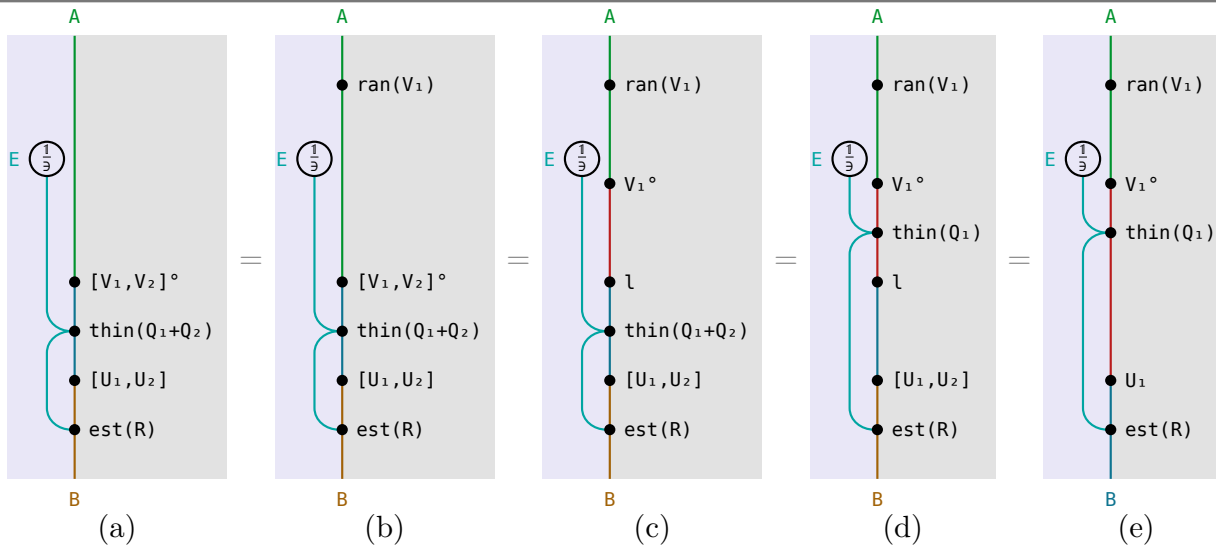
(a)	$\sqsubseteq h^\circ F(H^\circ) Q^\circ E (E \setminus (F(\frac{H}{\exists} \text{est}(R)) h R^\circ))$	$S (S \setminus R) \subseteq R$
(b)	$\sqsubseteq h^\circ F(H^\circ) Q^\circ F(\frac{H}{\exists} \text{est}(R)) h R^\circ$	$Q F(H) h \subseteq F(H) h R$ $\implies h^\circ F(H^\circ) Q^\circ \subseteq R^\circ h^\circ F(H^\circ)$
(c)	$\sqsubseteq R^\circ h^\circ F(H^\circ) F(\frac{H}{\exists} \text{est}(R)) h R^\circ$	

1



1 (a)	$R^\circ \ h^\circ \ F(H^\circ) \ F(\frac{H}{3} \ est(R)) \ h \ R^\circ$	$t(f \ g) = t(f) \ t(g)$
2 (b)	$= R^\circ \ h^\circ \ F(H^\circ \ \frac{H}{3} \ est(R)) \ h \ R^\circ$	$S^\circ \ \frac{S}{3} \ est(R) \subseteq R^\circ$
3 (c)	$\subseteq R^\circ \ h^\circ \ F(R^\circ) \ h \ R^\circ$	$\text{MonoAlg}(h, R^\circ)$
4 (d)	$\subseteq R^\circ \ R^\circ \ R^\circ$	$R^\circ \ R^\circ \subseteq R^\circ$
5 (e)	$\subseteq R^\circ \ R^\circ$	$R^\circ \ R^\circ \subseteq R^\circ$
6 (f)	$\subseteq R^\circ$	

$V_2 \ V_1^\circ = \emptyset \implies \frac{[V_1, V_2]^\circ}{3} \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ est(R) = (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(U_1) \ est(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(U_2) \ est(R))$ Proposition 9.1
 when V_1 and V_2 have disjoint ranges, the body over $[V_1, V_2]$ runs the V_1 problem on inputs V_1 reaches and the V_2 problem on inputs V_2 reaches



(a)	$\frac{[V_1, V_2]^\circ}{3} \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ est(R)$	empty off $\text{ran } V_1 \cup \text{ran } V_2$
(b)	$= (\text{ran}(V_1) \ \frac{[V_1, V_2]^\circ}{3} \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ est(R)) \cup (\text{ran}(V_2) \ \frac{[V_1, V_2]^\circ}{3} \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ est(R))$	$\frac{[V_1, V_2]^\circ}{3} = \frac{V_1^\circ}{3} P(\text{inl})$ on $\text{ran } V_1$
(c)	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ P(l) \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ est(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ P(r) \ \text{thin}(Q_1 + Q_2) \ P([U_1, U_2]) \ est(R))$	$P(\text{inl}) \ \text{thin}(Q_1 + Q_2) = \text{thin}(Q_1) P(\text{inl})$
(d)	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(l) \ P([U_1, U_2]) \ est(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(r) \ P([U_1, U_2]) \ est(R))$	P a relator, $\text{inl}[U_1, U_2] = U_1$
(e)	$= (\text{ran}(V_1) \ \frac{V_1^\circ}{3} \ \text{thin}(Q_1) \ P(U_1) \ est(R)) \cup (\text{ran}(V_2) \ \frac{V_2^\circ}{3} \ \text{thin}(Q_2) \ P(U_2) \ est(R))$	

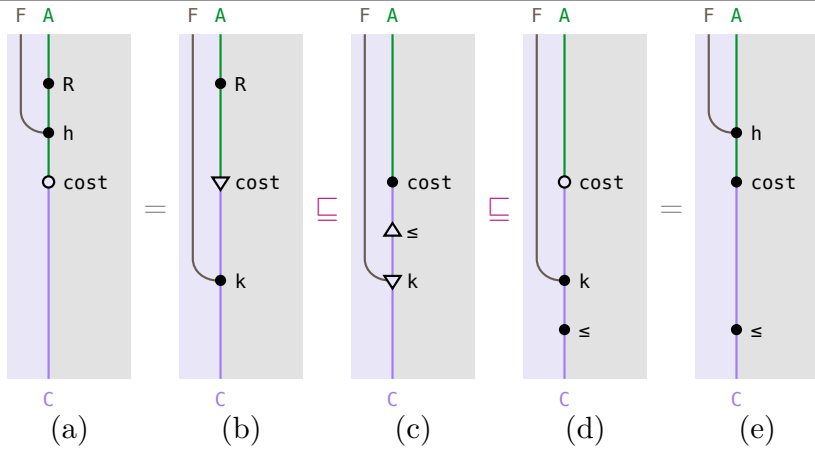
Map(cost) $\wedge$ R = cost $\leq$ cost $^\circ$ $\wedge$ h cost = F(cost) k $\wedge$ F($\leq$) k $\sqsubseteq$ k $\leq \implies$ MonoAlg(h,R) Proposition 9.2

if R compares two values by comparing their **costs** under $\leq$, and h then **cost** equals F(**cost**) then

a k monotonic on $\leq$, then h is monotonic on R

Map(cost) $\wedge$ R = cost $\leq$ cost $^\circ$ $\wedge$ h cost = F(cost) k $\wedge$ F($\leq$) k $\sqsubseteq$ k $\leq \implies$ F(R) h cost $\sqsubseteq$ h cost $\leq$

by the definition of R and shunting at the map **cost**, F(R)h $\sqsubseteq$ hR is F(R)h cost $\sqsubseteq$ h cost $\leq$, which the chain proves



(a)	F(R) h cost	h cost = F(cost) k
(b)	= F(R) F(cost) k	Map(f) $\wedge$ R = f S f $^\circ$ $\implies$ R f $\sqsubseteq$ f S
(c)	$\sqsubseteq$ F(cost) F($\leq$) k	F($\leq$) k $\sqsubseteq$ k $\leq$
(d)	$\sqsubseteq$ F(cost) k $\leq$	h cost = F(cost) k
(e)	= h cost $\leq$	

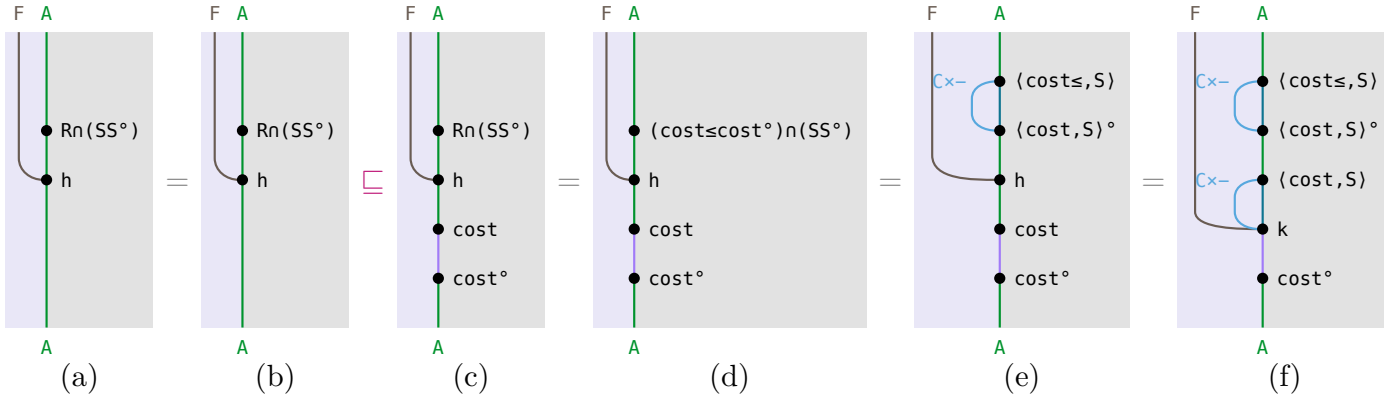
(9.1.1h)

	the condition	how it is discharged
1	F(R)h $\sqsubseteq$ hR Proposition 9.2, R $\triangleq$ cost $\leq$ cost $^\circ$, h cost = F(cost)k, F($\leq$) k $\sqsubseteq$ k $\leq$	monotonicity when the cost is itself a fold with a step k monotonic on $\leq$
2	F(Rn(H $^\circ$ H))h $\sqsubseteq$ hR Proposition 9.3, R $\triangleq$ cost $\leq$ cost $^\circ$, h cost = F((cost, H $^\circ$))k, F($\leq \times 1$) k $\sqsubseteq$ k $\leq$, H $^\circ$ simple;	monotonicity in context : k may also read the input the part was built from
3	QF(H)h $\sqsubseteq$ F(H)hR at Q $\triangleq$ F(U,V) Proposition 9.4, U, V preorders, F(U,R)h $\sqsubseteq$ hR, Vh $\sqsubseteq$ hR	both conditions at once, split along the two arguments of a bifunctor

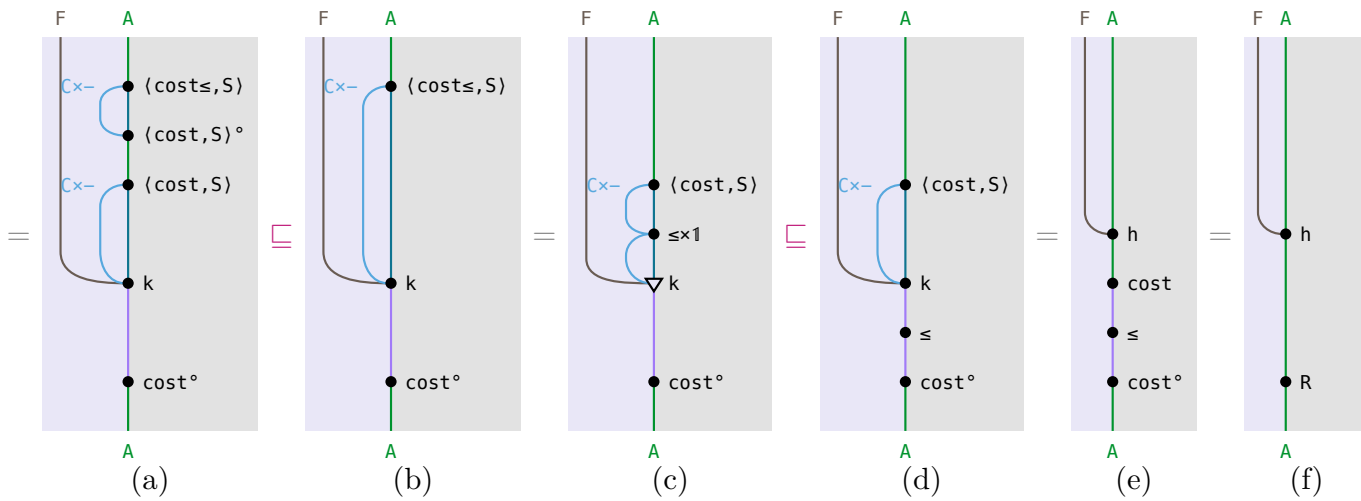
Map(cost) $\wedge$ Simple(S) $\wedge$ R = cost $\leq$ cost $^\circ$ $\wedge$ h cost = F((cost,S)) k $\wedge$ F($\leq \times 1$) k $\sqsubseteq$ Proposition 9.3

k $\leq \Rightarrow$ F(R $\cap$ (S S $^\circ$)) h $\sqsubseteq$ h R

improving each part by R within its S-context, then assembling by h, is below hR



(a)	$F(R \cap (S S^\circ)) h$	
(b)	$= F(R \cap (S S^\circ)) h$	Map(f) $\Rightarrow 1 \sqsubseteq f f^\circ$
(c)	$\sqsubseteq F(R \cap (S S^\circ)) h \text{ cost cost}^\circ$	$R = \text{cost} \leq \text{cost}^\circ$
(d)	$= F((\text{cost} \leq \text{cost}^\circ) \cap (S S^\circ)) h \text{ cost cost}^\circ$	$\langle X, Y \rangle \langle R, S \rangle^\circ = (X R^\circ) \cap (Y S^\circ)$
(e)	$= F(\langle \text{cost} \leq, S \rangle \langle \text{cost}, S \rangle^\circ) h \text{ cost cost}^\circ$	$h \text{ cost} = F(\langle \text{cost}, S \rangle) k$
(f)	$= F(\langle \text{cost} \leq, S \rangle \langle \text{cost}, S \rangle^\circ) F(\langle \text{cost}, S \rangle) k \text{ cost}^\circ$	$t(f g) = t(f) t(g)$



(a)	$= F(\langle \text{cost} \leq, S \rangle \langle \text{cost}, S \rangle^\circ \langle \text{cost}, S \rangle) k \text{ cost}^\circ$	
(b)	$\sqsubseteq F(\langle \text{cost} \leq, S \rangle) k \text{ cost}^\circ$	$\langle X, Y \rangle (R \times 1) = \langle X R, Y \rangle$ $t(f g) = t(f) t(g)$
(c)	$= F(\langle \text{cost}, S \rangle) F(\leq \times 1) k \text{ cost}^\circ$	$F(\leq \times 1) k \sqsubseteq k \leq$
(d)	$\sqsubseteq F(\langle \text{cost}, S \rangle) k \leq \text{cost}^\circ$	$h \text{ cost} = F(\langle \text{cost}, S \rangle) k$
(e)	$= h \text{ cost} \leq \text{cost}^\circ$	$R = \text{cost} \leq \text{cost}^\circ$
(f)	$= h R$	

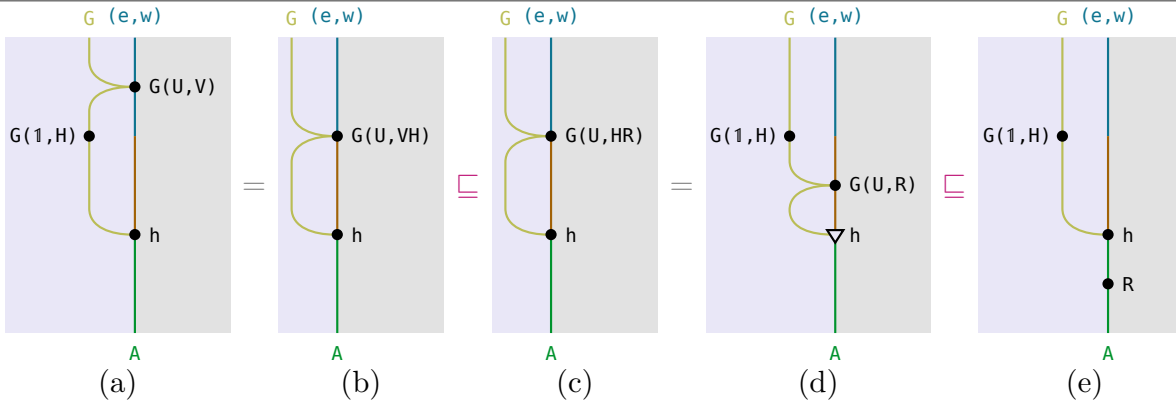
G(U,R) h $\sqsubseteq$ h R $\wedge$ V H $\sqsubseteq$ H R $\Rightarrow$ G(U,V) G(1,H) h $\sqsubseteq$ G(1,H) h R

Proposition 9.4

thinning the parts by U in the first argument and by V in the second (G(U,V)), then solving by H

and assembling by h, gives only what solving and assembling and then improving by R gives

(G(1,H)hR)



(a)	$G(U,V) G(1,H) h$	$F(R R', S S') = F(R, S) F(R', S')$
(b)	$= G(U, V H) h$	$V H \sqsubseteq H R$
(c)	$\sqsubseteq G(U, H R) h$	$F(R R', S S') = F(R, S) F(R', S')$
(d)	$= G(1,H) G(U,R) h$	$G(U,R) h \sqsubseteq h R$
(e)	$\sqsubseteq G(1,H) h R$	

§9.2. The string edit problem

- $(\exists a. o = \text{cpy}(a)) \vee (\exists a. o = \text{del}(a)) \vee (\exists a. o = \text{ins}(a))$
an edit operation copies, deletes or inserts one character
- $\text{FX} = 1 + (0p \times X)$
 $F(0p, X)$ is the empty sequence or one operation paired with an X
- $\text{edit} = ([\text{base}, \text{step}])$
 edit is the fold of $[\text{base}, \text{step}]$ over an edit sequence, returning the two strings it reconstitutes
- $\text{length} = ([\text{zero}, \pi_2 \text{ succ}])$
the fold of $[\text{zero}, \pi_2 \text{ succ}]$ counts the operations of a sequence
- $R = \text{length} \leq \text{length}^\circ$
 R relates es to fs when es is no longer than fs
- $V = \text{suffix}^\circ \times \text{suffix}^\circ$
 V relates two pairs of strings when each string of the first is a suffix of the matching string of the second
- $Q(\text{inl}(u), \text{inl}(u))$
 Q is the identity on the **base** summand
- $Q(\text{inr}(p), \text{inr}(q)) \iff V(\pi_2(p), \pi_2(q))$
on the **step** summand Q compares any two operations ($U \triangle T$) and the two pairs of strings by V
- $\text{empty}(p, q) \iff p = q \wedge p = ([], [])$
 empty is the coreflexive holding only at the pair of two empty strings
- $q \in \text{unstep}(p) \implies \text{step}(\text{inr}(q)) = p$
every decomposition unstep returns is sent back to its input by step

(9.2a)

x	$\alpha(x)$
1 $\text{inl}(u)$	$\text{wrap}(u)$
2 $\text{inr}((o, es))$	$[o]\#es$

(9.2b)

x	$[\text{base}, \text{step}](x)$
1 $\text{inl}(u)$	$([], [])$
2 $\text{inr}((\text{cpy}(a), (xs, ys)))$	$([a]\#xs, [a]\#ys)$
3 $\text{inr}((\text{del}(a), (xs, ys)))$	$([a]\#xs, ys)$
4 $\text{inr}((\text{ins}(a), (xs, ys)))$	$(xs, [a]\#ys)$

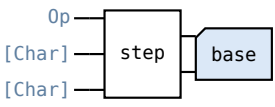
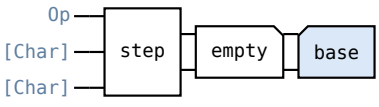
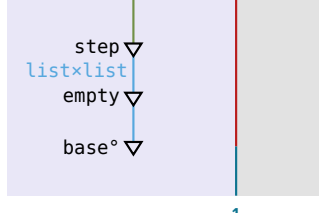
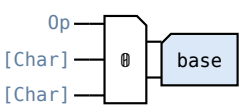
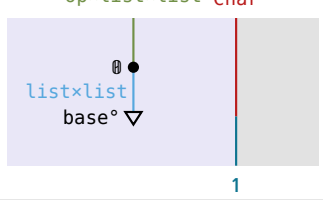
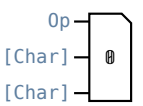
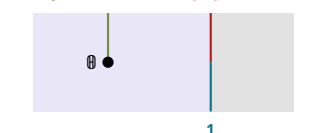
(9.2c)

p	$\text{unstep}(p)$
1 $([], [])$	$[]$
2 $([a]\#xs, [])$	$[(\text{del}(a), (xs, []))]$
3 $([], [b]\#ys)$	$[(\text{ins}(b), ([], ys))]$
4 $([a]\#xs, [b]\#ys)$	if $a = b$ then $[(\text{cpy}, xs, ys)]$ else $[(\text{del}, xs, [b]\#ys), (\text{ins}, [a]\#xs, ys)]$

(9.2d)

circuit	Hinze–Marsden
length is a map, so entire: $1 \sqsubseteq \text{length} \text{ length}^\circ$	
$\alpha \text{ length} = F(\text{length})[\text{zero}, \pi_2 \text{ succ}] : \text{length}^\Delta([\text{zero}, \pi_2 \text{ succ}]) \triangleq \text{edit}$	
length is a map, so simple, and F is monotonic: $F(\text{length}^\circ)F(\text{length}) \sqsubseteq F(1) = 1$	
SUCC is monotonic on $\leq$: $F(\leq)[\text{zero}, \pi_2 \text{ succ}] \sqsubseteq [\text{zero}, \pi_2 \text{ succ}] \leq$	
$F(\text{length})[\text{zero}, \pi_2 \text{ succ}] = \alpha \text{ length}$ again	
$R \triangleq \text{length} \leq \text{length}^\circ \triangleq \text{edit}$	

$\text{edit } V^\circ \sqsubseteq R^\circ \text{ edit}$ (9.2h) cutting a front off either string an edit sequence produces ($\text{edit } V^\circ$) leaves a pair that a sequence no longer produces ($R^\circ \text{ edit}$)	
circuit	Hinze–Marsden
$=$ $V \triangleq \text{suffix}^\circ \times \text{suffix}^\circ$, and $\times$ preserves composition	
$\sqsubseteq$ $\text{edit } (\text{suffix} \times 1) \sqsubseteq R^\circ \text{ edit}$: drop the operation that produced the head, or weaken its cpy to an ins. Proved by induction at suffix directly, not through $\text{suffix} = \text{tail}^*$ and $BA \sqsubseteq CB \implies BA^* \sqsubseteq C^*B$ as the book does	
$\sqsubseteq$ $\text{edit } (1 \times \text{suffix}) \sqsubseteq R^\circ \text{ edit}$, the mirror image: cpy weakens to a del	
$\sqsubseteq$ R° is transitive, $\leq$ being so	

$\text{step } \text{base}^\circ = \emptyset$ (9.2i) no pair of strings is both a result of step and the result $([], [])$ of base	
circuit	Hinze–Marsden
	
 base returns only $([], [])$, so $\text{base}^\circ = \text{empty } \text{base}^\circ$	
 $\text{step } \text{empty} = \emptyset$: cpy and del put a character on the left string, ins one on the right	
 $\emptyset$ composed with anything is $\emptyset$	

1	$\text{mle}(xs, ys) = \text{head}(\text{column}(xs, ys))$, $\text{column}(xs, ys) = [\text{mle}(u, ys) \mid u \leftarrow \text{tails}(xs)]$ $= \text{column}(xs) = ([\text{fstcol}(xs), \text{nextcol}(xs)])$, $\text{fstcol} = \text{list}(\text{del}) \text{ tails}$ the tabulation: $\text{mle}(xs, ys)$ needs $\text{mle}(u, v)$ for every tail u of xs and v of ys , so the columns are built right to left
2	$\text{column}(xs, [b] \# ys) = \text{nextcol}(xs, (b, \text{column}(xs, ys)))$ $= \text{nextcol}(xs)(b, us) = ([\text{base}(b, \text{last}(us)), \text{step}(b)])(xus)$, $xus = \text{zip}(xs, \text{zip}(\text{init}(us), \text{tail}(us)))$ each column is a fold built bottom to top, over xs zipped with the adjacent pairs of the column to its right
3	$\text{base}(b, u) = [[\text{ins}(b)] \# u]$ $= \text{step}(b)((a, (u, v)), ws) = (a \rightarrow b \rightarrow [[\text{cpy}(a)] \# v] \# ws, [\text{bmin}(R)([\text{del}(a)] \# w, [\text{ins}(b)] \# u)] \# ws)$ $w = \text{head}(ws)$; these base , step are not edit 's. An entry depends on the one below it (a delete), the one to its right (an insert), and the one below that (a copy) — quadratic in the two lengths

§9.3. Optimal bracketing

tree $A ::= \text{tip } A \mid \text{bin } (\text{tree } A, \text{tree } A)$, $FX = A + X^2$, so $F(R) = 1 + R^2$; $h \triangleq [\text{tip}, \text{bin}]$, $\text{flatten} \triangleq ([\text{wrap}, \text{cat}]) : \text{tree } A \rightarrow \text{list}^+ A$, $H = \text{flatten}^\circ$. $\langle \text{cost}, \text{size} \rangle \triangleq ([\text{opt}, \text{opb}])$, $\text{opt} \triangleq (\text{zero}, \text{st})$, $\text{opb } ((cx, sx), (cy, sy)) = (\text{cb } (sx, sy) + cx + cy, \text{sb } (sx, sy))$. sb associative, so $\text{size} = \text{flatten } \text{sz}$ for a map sz $R \triangleq \text{cost} \leq \text{cost}^\circ$, $g \triangleq [\text{zero}, (1 \times \text{sz})^2 \text{ opb } \pi_1]$, single the coreflexive on singleton lists. $\text{splits} \triangleq (\text{inits}^+, \text{tails}^+)$ zip, an implementation of $\frac{\text{cat}^\circ}{\text{zip}}$; $\text{array} \triangleq \text{inits } \text{list}(\text{row})$, $\text{row} \triangleq \text{tails } \text{list}(\text{mct})$, $\text{col} \triangleq \text{inits } \text{list}(\text{mct})$. $\text{mix} \triangleq \text{zip } \text{list}(\text{bin}) \text{ minlist}(R)$, $\text{next} \triangleq (\pi_1, \text{mix}) \text{ snoc}$, $\text{process} \triangleq ((\text{tip } \text{wrap}) \times 1) \text{ loop}(\text{next})$.	(9.3a)
---	--------

$\text{Assoc}(\text{sb}) \Rightarrow F(\langle \text{cost}, \text{flatten} \rangle) \text{ g} = [\text{tip}, \text{bin}] \text{ cost}$ (9.5) pairing each subtree with its cost and its flattening and then applying g gives what building the tree by $[\text{tip}, \text{bin}]$ and taking its cost gives	
circuit	Hinze–Marsden
$F(\text{tree}(A)) \rightarrow \boxed{F(\langle \text{cost}, \text{flatten} \rangle)} \rightarrow F(\text{Int} \times \text{list}^+(A)) \rightarrow \boxed{\text{g}} \rightarrow \text{Int}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} \text{l} \rightarrow A \rightarrow \text{zero} \\ \text{r} \rightarrow \text{tree}(A) \rightarrow \left\{ \begin{array}{l} \text{cost} \\ \text{flatten} \rightarrow \text{list}^+(A) \rightarrow \text{sz} \end{array} \right\} \\ \text{r} \rightarrow \text{tree}(A) \rightarrow \left\{ \begin{array}{l} \text{cost} \\ \text{flatten} \rightarrow \text{list}^+(A) \rightarrow \text{sz} \end{array} \right\} \end{array} \right\} \rightarrow \text{opb} \rightarrow \text{Int}$ definition of g; coproducts and products $\triangleq \text{flatten}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} \text{l} \rightarrow A \rightarrow \text{zero} \\ \text{r} \rightarrow \text{tree}(A) \rightarrow \left\{ \begin{array}{l} \text{cost} \\ \text{size} \end{array} \right\} \\ \text{r} \rightarrow \text{tree}(A) \rightarrow \left\{ \begin{array}{l} \text{cost} \\ \text{size} \end{array} \right\} \end{array} \right\} \rightarrow \text{opb} \rightarrow \text{Int}$ $\text{flatten sz} = \text{size}$, since sb is associative $\triangleq \text{flatten}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} \text{l} \rightarrow A \rightarrow \text{zero} \\ \text{r} \rightarrow \text{tree}(A) \rightarrow \text{bin} \rightarrow \text{tree}(A) \rightarrow \left\{ \begin{array}{l} \text{cost} \rightarrow \text{Int} \\ \text{size} \rightarrow S \end{array} \right\} \end{array} \right\} \rightarrow \text{opb} \rightarrow \text{Int}$ $\langle \text{cost}, \text{size} \rangle \triangleq ([\text{opt}, \text{opb}])$, at a node $\text{bin}(\text{cost}, \text{size}) = \langle \text{cost}, \text{size} \rangle^2 \text{ opb}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} \text{l} \rightarrow A \rightarrow \text{tip} \rightarrow \text{tree}(A) \rightarrow \text{cost} \\ \text{r} \rightarrow \text{tree}(A) \rightarrow \text{bin} \rightarrow \text{tree}(A) \rightarrow \text{cost} \end{array} \right\} \rightarrow \text{Int}$ $\text{tip cost} = \text{zero}$; products, $\langle \text{cost}, \text{size} \rangle \pi_1 = \text{cost}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} \text{l} \rightarrow A \rightarrow \text{tip} \\ \text{r} \rightarrow \text{tree}(A) \rightarrow \text{bin} \end{array} \right\} \rightarrow \text{tree}(A) \rightarrow \text{cost} \rightarrow \text{Int}$ coproducts, $[\text{tip cost}, \text{bin cost}] = [\text{tip}, \text{bin}] \text{ cost}$	

$F(\leq \times 1) \ g \sqsubseteq g \leq$ (9.6)	
raising the cost of either subtree ($\leq \times 1$ in both slots) and then applying g returns a value at least what g returns on the costs before raising ($g \leq$)	
circuit	Hinze–Marsden
$F(\text{Int} \times \text{list}^+(A)) \rightarrow \boxed{F(\leq \times 1)} \rightarrow \boxed{g} \rightarrow \text{Int}$	
$A + \text{Int} \times \text{list}^+(A) \times \text{Int} \times \text{list}^+(A) \rightarrow \text{Int}$ $=$ definition of $g \triangleq \text{flatten}$	
$A + \text{Int} \times \text{list}^+(A) \times \text{Int} \times \text{list}^+(A) \rightarrow \text{Int}$ $\sqsubseteq$ definition of opb , and $+$ monotonic	
$A + \text{Int} \times \text{list}^+(A) \times \text{Int} \times \text{list}^+(A) \rightarrow \text{Int}$ $\sqsubseteq$ $\leq$ reflexive, so $\text{zero} \sqsubseteq \text{zero} \leq$; coproducts	
$F(\text{Int} \times \text{list}^+(A)) \rightarrow \boxed{g} \rightarrow \boxed{\leq} \rightarrow \text{Int}$ $=$ definition of $g \triangleq \text{flatten}$	

$$\text{mct} = (\text{single} \rightarrow \text{head tip}, (\text{init col}, \text{tail row}) \text{ mix}), \text{ Assoc}(\text{sb}) \Rightarrow (\mu X : \frac{[\text{wrap}, \text{cat}]^\circ}{3} P([\text{tip}, (X \times X) \text{ bin}]) \text{ est}(\text{R})) \sqsubseteq \frac{\text{flatten}^\circ}{3} \text{ est}(\text{R}) \quad (9.3b)$$

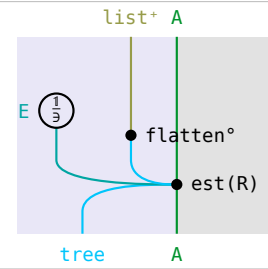
a least-cost bracketing of $a_1 \oplus \dots \oplus a_n$ is read off an array holding one best tree per non-empty segment, each entry built from the column to its left and the row below it

circuit

$$\text{list}^+(A) \xrightarrow{\frac{1}{3}} P(\text{list}^+(A)) \xrightarrow{E(\text{flatten}^\circ)} P(\text{tree}(A)) \xrightarrow{\text{est}(\text{R})} \text{tree}(A)$$

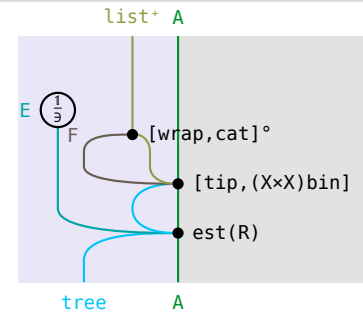
the specification $\triangleq \text{flatten}$

Hinze–Marsden



$$\text{list}^+(A) \xrightarrow{\frac{1}{3}} P(\text{list}^+(A)) \xrightarrow{E([\text{wrap}, \text{cat}]^\circ)} P(\text{list}^+(A)) \xrightarrow{P([\text{tip}, (X \times X) \text{ bin}])} P(\text{tree}(A)) \xrightarrow{\text{est}(\text{R})} \text{tree}(A)$$

split the list in every way, bracket both halves, join. The condition is monotonicity **in context**,
 $\sqsubseteq F(\text{Rn}(\text{flatten} \text{ flatten}^\circ)) \text{ h} \sqsubseteq \text{hR}$ — only trees with the same flattening are compared — which is Proposition 9.3 at $\text{H}^\circ = \text{flatten}$, a map, with (9.5) $[\text{tip}, \text{bin}] \text{ cost} = (1 + (\text{cost}, \text{flatten})^2) \text{g}$ (the cost of a node reads only the cost and the flattening of its two subtrees) and (9.6) $(1 + (\leq \times 1)^2) \text{g} \sqsubseteq \text{g}$ (g monotonic on $\leq$ in its two cost arguments)

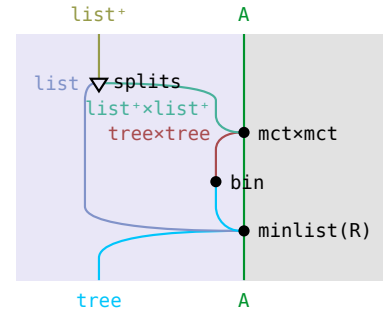


$$\text{list}^+(A) \xrightarrow{\frac{1}{3}} P(\text{list}^+(A)) \xrightarrow{E(\text{cat}^\circ)} P(\text{list}^+(A)^2) \xrightarrow{P((X \times X) \text{ bin})} P(\text{tree}(A)) \xrightarrow{\text{est}(\text{R})} \text{tree}(A)$$

$\equiv$ Proposition 9.1: **wrap** and **cat** have disjoint ranges, and **single** is the coreflexive on singleton lists, where **wrap** returns

$$\text{list}^+(A) \xrightarrow{\text{splits}} [\text{list}^+(A)^2] \xrightarrow{\text{list}((\text{mct} \times \text{mct}) \text{ bin})} [\text{tree}(A)] \xrightarrow{\text{minlist}(\text{R})} \text{tree}(A)$$

$\sqsubseteq$ $\text{splits} \triangleq \langle \text{inits}^+, \text{tails}^+ \rangle$ zip implements $\frac{\text{cat}^\circ}{3}$ and $\text{minlist}(\text{R})$ implements $\text{est}(\text{R})$. Exponential, since the segments of one list overlap



$$\text{mct} = (\text{single} \rightarrow \text{head tip}, (\text{init col}, \text{tail row}) \text{ mix}) \quad (9.7)$$

$\equiv$ the tabulation: $\text{mct } \text{xs}$ is needed for every non-empty segment xs , so the values are held as an array of rows, $\text{array} \triangleq \text{inits list}(\text{row}), \text{row} \triangleq \text{tails list}(\text{mct}), \text{col} \triangleq \text{inits list}(\text{mct}), \text{mix} \triangleq \text{zip list}(\text{bin}) \text{ minlist}(\text{R})$

$$\text{col} = (\text{single} \rightarrow \text{head tip wrap}, (\text{init col}, \text{tail row}) \text{ next}) \quad (9.8)$$

$$\text{cons col} = (1 \times \text{array}) \text{ process} \quad (9.9)$$

$$\equiv \text{row} = (\text{single} \rightarrow \text{head tip wrap}, (\text{mct}, \text{tail row}) \text{ cons}) \quad (9.10)$$

a column extends the column to its left, a row the row below it; (9.9) is (9.8) rewritten as a loop, $\text{next} \triangleq \langle \pi_1, \text{mix} \rangle \text{ snoc}, \text{process} \triangleq ((\text{tip wrap}) \times 1) \text{ loop}(\text{next})$

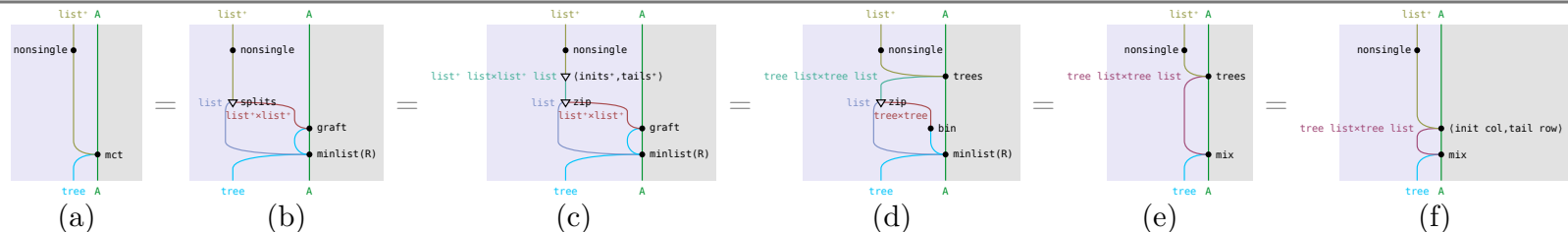
$$\text{array} = ([\text{fstcol}, \text{addcol}]), \text{fstcol} \triangleq \text{tip wrap wrap}$$

$$\equiv \text{addcol} \triangleq \langle \pi_1 \text{ tip wrap}, \text{step} \rangle \text{ cons}, \text{step} \triangleq \langle \text{process tail}, \pi_2 \rangle \text{ zip list}(\text{cons})$$

the program: one fold building the array column by column, cubic in the length of the input

$$\text{nonsingle mct} = \text{nonsingle } (\text{init col}, \text{tail row}) \text{ mix} \quad (9.7)$$

on a list of two or more elements, mct gives what mix gives on the column of its **init** and the row of its **tail**

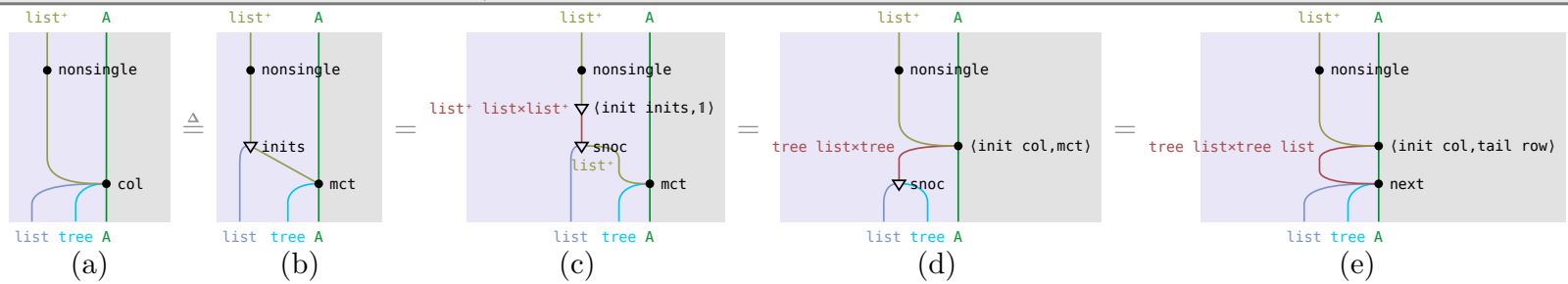


(a)	<code>nonsingle mct</code>	$\text{mct} \triangleq \text{flatten}$
(b)	$\equiv \text{nonsingle splits list(graft) minlist(R)}$	$\text{splits} \triangleq \text{flatten}$
(c)	$\equiv \text{nonsingle } \langle \text{inits}^+, \text{tails}^+ \rangle \text{ zip list(graft) minlist(R)}$	$\text{graft, trees; zip list}(f \times g) = (\text{list}(f) \times \text{list}(g)) \text{ zip}$
(d)	$\equiv \text{nonsingle trees zip list(bin) minlist(R)}$	$\text{mix} \triangleq \text{zip list(bin) minlist(R)} \triangleq \text{flatten}$
(e)	$\equiv \text{nonsingle trees mix}$	$\text{inits}^+ = \text{init inits}, \text{tails}^+ = \text{tail tails}; \text{row, col} \triangleq \text{flatten}$
(f)	$\equiv \text{nonsingle } \langle \text{init col}, \text{tail row} \rangle \text{ mix}$	

nonsingle col = nonsingle (init col,tail row) next

(9.8)

on a list of two or more elements, the column is what **next** makes of the column of its **init** and the row of its **tail**

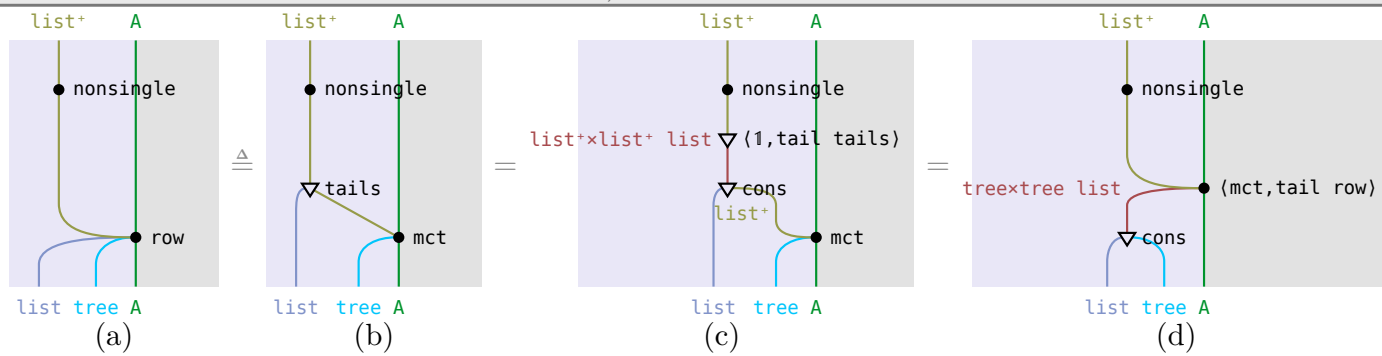


(a)	nonsingle col	col $\triangleq$ flatten
(b)	$\triangleq$ nonsingle inits list(mct)	inits=(init inits,1) snoc on non-singletons
(c)	$=$ nonsingle (init inits,1) snoc list(mct)	snoc list(f)=(list(f) $\times$ f) snoc; col
(d)	$=$ nonsingle (init col,mct) snoc	(9.7) — (9.7); next $\triangleq$ (π_1 ,mix) snoc $\triangleq$ flatten
(e)	$=$ nonsingle (init col,tail row) next	

nonsingle row = nonsingle (mct,tail row) cons

(9.10)

on a list of two or more elements, the row is **mct** of the whole list consed onto the row of its **tail**



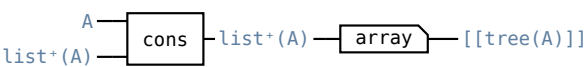
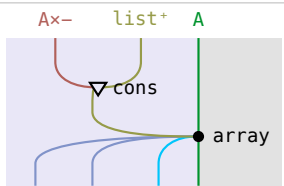
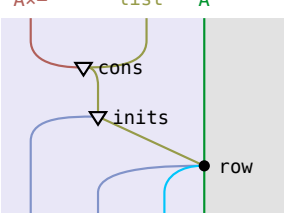
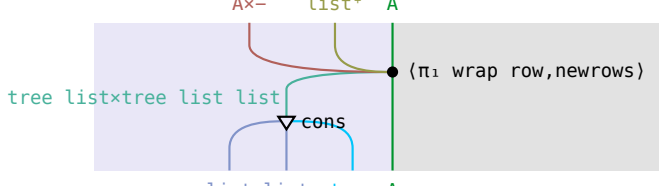
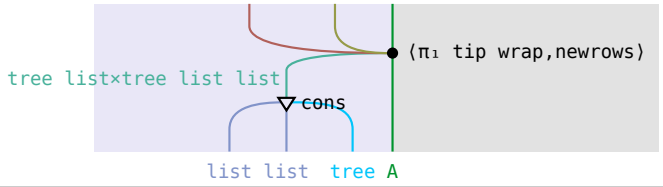
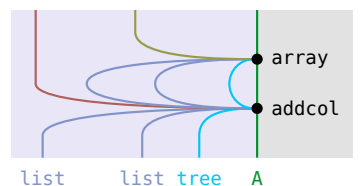
(a)	nonsingle row	row $\triangleq$ flatten
(b)	$\triangleq$ nonsingle tails list(mct)	tails=(1,tail tails) cons on non-singletons
(c)	$=$ nonsingle (1,tail tails) cons list(mct)	cons list(f)=(f $\times$ list(f)) cons; row
(d)	$=$ nonsingle (mct,tail row) cons	

cons col = (1 × array) process (9.9) the column of a consed onto x is what process makes of a and the array of x	
circuit	Hinze–Marsden
= Exercise 9.13 with $k \triangleq \text{cons}$, $g \triangleq \text{tip}$, $h \triangleq \text{wrap}$, $f \triangleq \text{next}$: its two equations are (9.8) — (9.8) — at $a:[b]$ and at $a:(u++[b])$, with $[a] \text{ col} = [\text{tip}(a)]$	
= products	
= definition of process	

tops = (1 × array) process tail (9.3c) the trees at the top of the new rows are what process makes of a and the array of x , less its first	
circuit	Hinze–Marsden
= definition of $\text{tops} \triangleq \text{tic list(mct)}$, our name	
tail list(f)=list(f) tail; definition of col	
= (9.9) — (9.9)	

rests = π_2 array (9.3d) the rest of each new row is the array of x	
circuit	Hinze–Marsden
definition of $\text{rests} \triangleq \text{tic list}(\text{tail row})$, our name	
$\text{tic list}(\text{tail}) = \pi_2 \text{ inits}$; definition of array	

newrows = (1 × array) step (9.3e) the rows of the array of a:x below its first are what step makes of a and the array of x	
circuit	Hinze–Marsden
definition of $\text{newrows} \triangleq \text{tic list}(\text{row})$, our name	
(9.10) — (9.10) — on the non-singleton tic lists	
$\text{list}(f, g) = (\text{list}(f), \text{list}(g))$ zip; products; definitions of tops and rests	
(9.3c) and (9.3d); products; definition of step	

$\text{cons array} = (1 \times \text{array}) \text{ addcol}$ (9.3f) the array of a consed onto x is what addcol makes of a and the array of x	
circuit	Hinze–Marsden
	
$= \text{list}^+(A) \text{ cons } \text{list}^+(A) \text{ inits } [\text{list}^+(A)] \text{ list}(\text{row}) [[\text{tree}(A)]]$ definition of array	
$= \text{cons inits} = (\pi_1 \text{ wrap, tic}) \text{ cons, abbreviating cons inits tail by tic}$	
$= \text{list}^+(A) \text{ wrap } \text{list}^+(A) \text{ row } [[\text{tree}(A)]] \text{ cons } [[\text{tree}(A)]]$ $\text{cons list}(f) = (f \times \text{list}(f)) \text{ cons; definition of newrows}$	
$= \text{list}^+(A) \text{ wrap } \text{list}^+(A) \text{ row } [[\text{tree}(A)]] \text{ cons } [[\text{tree}(A)]]$ $\text{wrap row} = \text{tip wrap}$	
$= \text{list}^+(A) \text{ array } \text{addcol} [[\text{tree}(A)]]$ (9.3e); products; definition of addcol	

§9.4. Data compression

(9.4a)

$[A] ::= \text{nil} | \text{snoc } ([A], A), \text{ list}^+ A ::= \text{wrap } A | \text{snoc } (\text{list}^+ A, A), \text{ String} = [\text{Char}], \text{ Code} ::= \text{sym } \text{Char} | \text{ptr } (\text{String}, \text{String}^+), F(\text{Code}, \text{String}) = 1 + (\text{String} \times \text{Code}), \alpha \triangleq [\text{nil}, \text{snoc}].$

$\text{decode} \triangleq ([\text{nil}, \text{extend}]) : [\text{Code}] \rightarrow \text{String}, \text{ extend } (xs, \text{sym } a) = xs \# [a], \text{ extend } (xs, \text{ptr } (ys, zs)) = xs \# zs \text{ when } ys \# zs \text{ is a proper prefix of } xs \# zs; H = \text{decode}^\circ.$

$\text{size} \triangleq ([\text{zero}, \text{distr } [1 \times c, 1 \times p] \text{ plus}]) \text{ with } c, p \text{ the constant costs of a symbol and a pointer; } R \triangleq \text{size} \leq \text{size}^\circ, Q \triangleq F(\tau + \tau, \text{prefix}^\circ) = 1 + (\text{prefix}^\circ \times (\tau + \tau)), \text{ the two } \tau \text{ on symbols and on pointers.}$

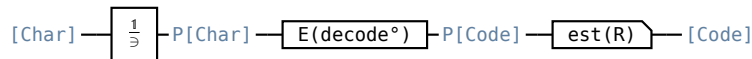
$\text{lrt } ws = \text{est}(\text{prefix}^\circ \times (\tau + \tau)) \{(xs, (ys, zs)) | xs \# zs = ws, ys \# zs \text{ a proper prefix of } ws\}, \text{ the longest repeated tail; reduce } (ws \# [a]) = (zs \# [] \rightarrow [(ws, \text{sym } a), (xs, \text{ptr } (ys, zs))], [(ws, \text{sym } a)]) \text{ with } (xs, (ys, zs)) = \text{lrt } (ws \# [a]).$

$$0 \leq c \wedge 0 \leq p \Rightarrow (\mu X : \frac{[\text{nil}, \text{extend}]^\circ}{3} \text{thin}(Q) P([\text{nil}, (X \times 1) \text{snoc}]) \text{est}(R)) \sqsubseteq \frac{\text{decode}^\circ}{3} \text{est}(R), \quad (9.4b)$$

$$\text{encode} = (\text{null} \rightarrow \text{nil}, \text{reduce list}((\text{encode} \times 1) \text{snoc}) \text{minlist}(R))$$

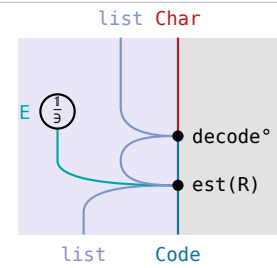
a smallest code sequence decoding to the given string is built from the right, each step emitting the last character as a symbol or ending with a pointer back into what has already been decoded

circuit



the specification — $\triangleq$ decode

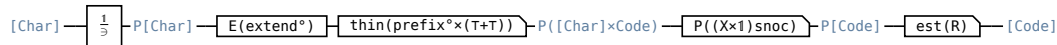
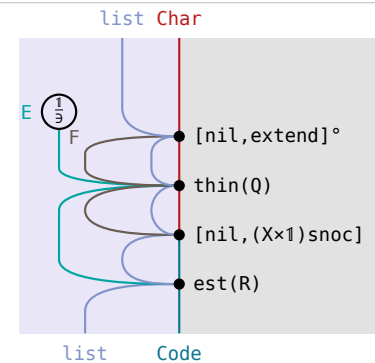
Hinze–Marsden



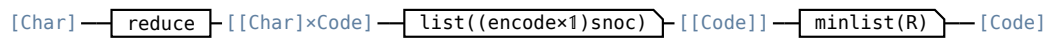
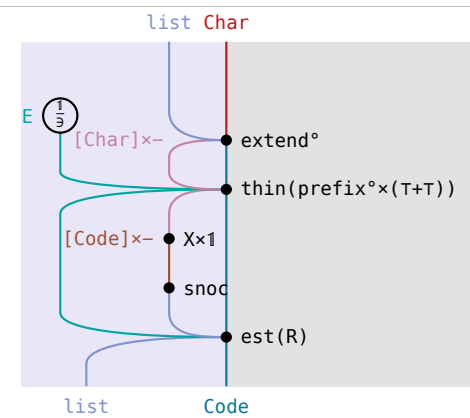
at $Q \triangleq F(T+T, \text{prefix}^\circ) = 1 + (\text{prefix}^\circ \times (T+T))$, the two T on symbols and on pointers.

Monotonicity $F(R) \alpha \sqsubseteq \alpha R$ is routine, the two costs being constants: $(T+T)[c, p] = [c, p]$.

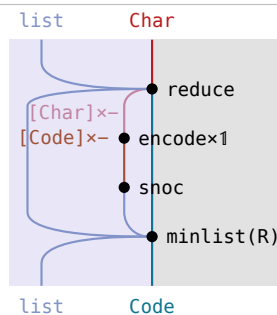
Proposition 9.4 splits the thinning condition in two — $F(T+T, R) \alpha \sqsubseteq \alpha R$, and $\text{decode prefix} \sqsubseteq R^\circ \text{decode}$, for which $\text{decode init} \sqsubseteq R^\circ \text{decode}$ is enough: dropping the last character of the output shortens or removes the last code element and never raises the cost. So between a symbol and a pointer nothing can be decided in advance, and between two pointers the longer match wins



Proposition 9.1: nil and extend have disjoint ranges. The decompositions of one string are $\frac{\text{extend}^\circ}{3} (ws\#[a]) = \{(ws, \text{sym } a)\} \cup \{(xs, \text{ptr } (ys, zs)) \mid xs\#zs = ws\#[a], ys\#zs \text{ a proper prefix of } ws\}$ — take the last character as a symbol, or end with a pointer



reduce implements $\frac{\text{extend}^\circ}{3} \text{thin}(\text{prefix}^\circ \times (T+T))$: thinning leaves at most two, the symbol and the pointer of the longest repeated tail lrt . Again exponential; the book gives no tabulation for it

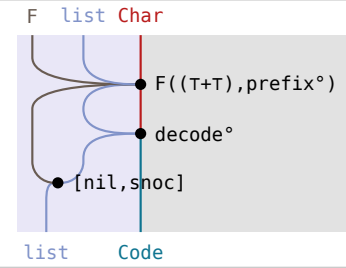
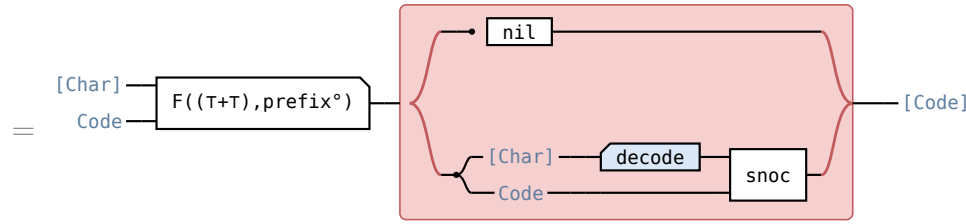
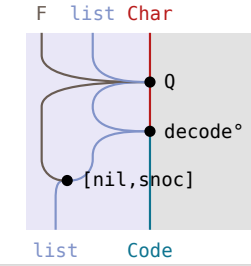
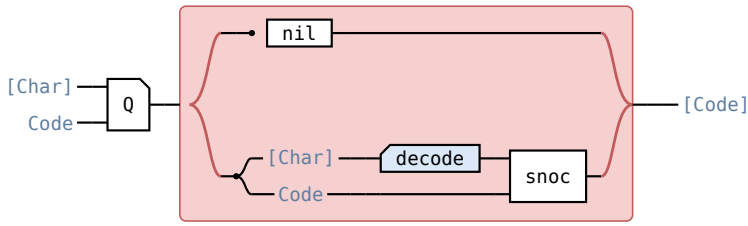


$$\theta \leq c \wedge \theta \leq p \Rightarrow Q \text{ F}(\text{decode}^\circ) [\text{nil}, \text{snoc}] \sqsubseteq \text{F}(\text{decode}^\circ) [\text{nil}, \text{snoc}] R \quad (9.4c)$$

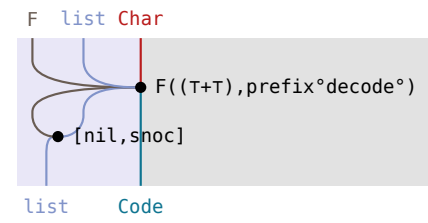
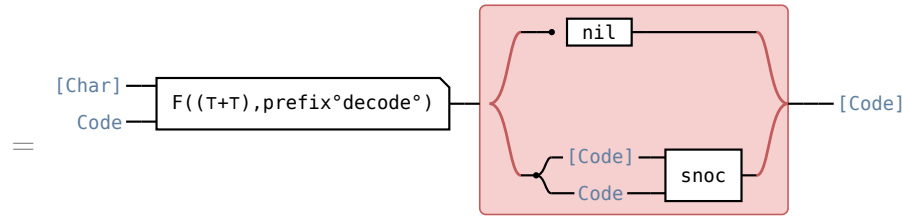
every code sequence built from a decomposition Q puts above a given one — a code element of the same kind, a longer front string — costs at least as much as one built from the given one

circuit

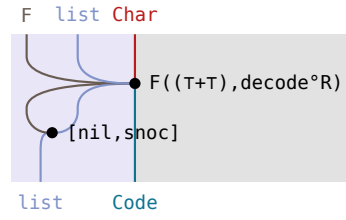
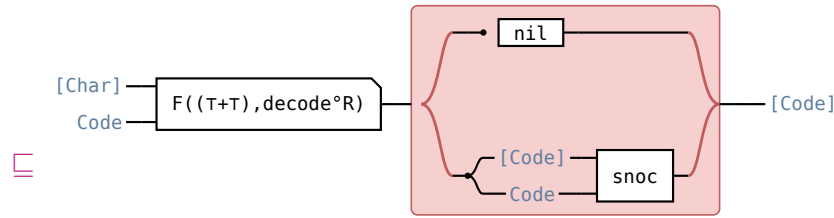
Hinze–Marsden



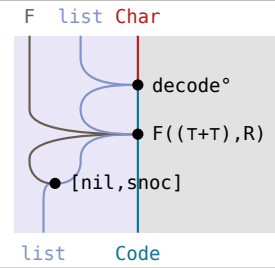
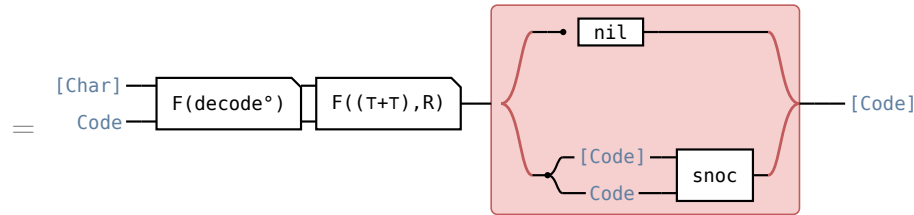
$Q \triangleq 1 + (\text{prefix}^\circ \times (T+T))$ is $F(T+T, \text{prefix}^\circ) \triangleq \text{decode}$



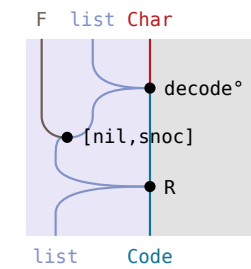
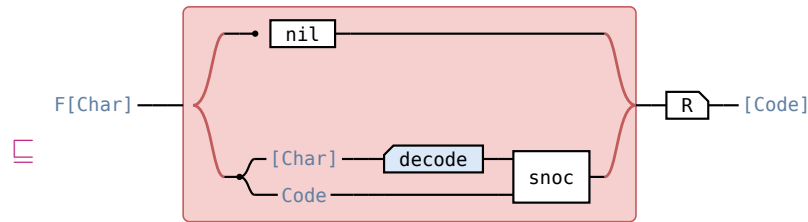
the bifunctor F preserves composition: $F(U, \text{prefix}^\circ) F(1, \text{decode}^\circ) = F(U, \text{prefix}^\circ \text{decode}^\circ)$



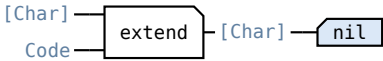
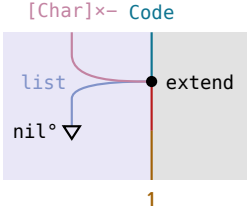
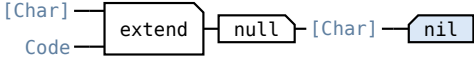
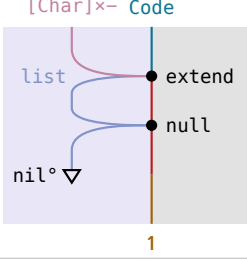
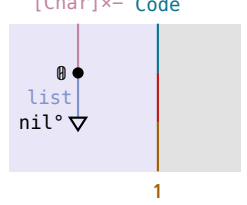
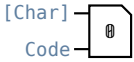
Proposition 9.4's second condition $\text{prefix}^\circ \text{decode}^\circ \sqsubseteq \text{decode}^\circ R$: dropping the last **sym**, or shortens or drops the last pointer. Proved at **prefix** directly by induction, not through **init**



F preserves composition: $F(U, \text{decode}^\circ R) = F(1, \text{decode}^\circ) F(U, R)$



Proposition 9.4's first condition $F(T+T, R) \alpha \sqsubseteq \alpha R$, left as an exercise in the book: **snoc** adds the cost of the last element to both sides, and $[c, p](T+T) = [c, p]$

extend nil[°] = 0 (9.4d) no string is both a result of extend and the result [] of nil	
circuit	Hinze–Marsden
	
 nil returns only [], so $\text{nil}^\circ = \text{null } \text{nil}^\circ$	
 $\text{extend } \text{null} = 0$: a symbol ends the string with a character, a pointer with its non-empty ZS	
 0 composed with anything is 0	